Understanding Neural Networks from the Ground Up: A NumPy-Only Implementation Compared with a Framework Baseline - Update 3

Priyo Raihan ID: 2311692042 | Sheanthea Akter ID: 2413587642 | Marzia Anjum ID: 2412117042 | Mustofa Nuruzzaman ID: 2231929642 | Sayema Tasnim Aroni ID: 2413241642

CSE 299, Section 3 — Group 4

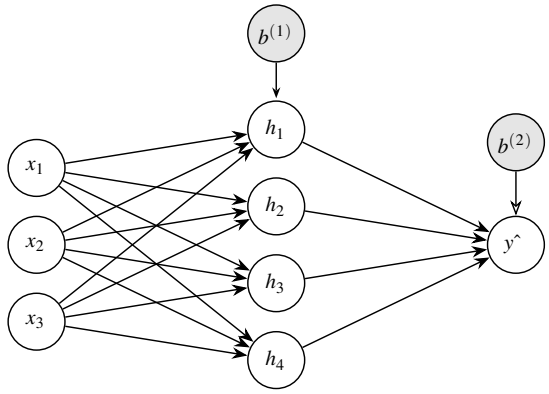


Fig. 1. The $3 \to 4 \to 1$ network (hidden ReLU, output sigmoid). Shaded nodes are biases.

***Abstract*— Neural networks require robust loss metrics and trainable bias terms to effectively approximate non-linear decision boundaries. Building upon our initial single-neuron prototype, this second progress update introduces explicit bias vectors and Binary Cross-Entropy (BCE) loss into a 2-layer neural network implemented strictly in NumPy. Using three standardized features from the Wisconsin Breast Cancer dataset, we derive backpropagation gradients for both weights and biases. Experimental results demonstrate smooth convergence over 100 epochs, reducing mean BCE loss from 0.4053 to 0.1442.**

I. Introduction

In our baseline setup, the model operated without explicit bias vectors and used a basic error formulation. While a single-neuron structure illustrates fundamental forward execution, real-world classification tasks demand non-linear hidden representations, flexible offsets (biases), and proper probabilistic loss formulations [1].

In this update, we upgrade our implementation to a multi-layer perceptron architecture ($3 \to 4 \to 1$) using the Wisconsin Breast Cancer (Diagnostic) dataset [2]. We integrate explicit bias parameters ($\mathbf{b}^{(1)}$, $b^{(2)}$) across layers and implement BCE loss alongside Mean Squared Error (MSE) to assess prediction confidence and training behavior during backpropagation.

II. Theory

A. Forward Propagation with Bias

Fig. 1 shows the network. For input $\mathbf{x} \in \mathbb{R}^3$, the hidden layer uses weights $W^{(1)} \in \mathbb{R}^{3\times 4}$, bias $\mathbf{b}^{(1)} \in \mathbb{R}^4$ and ReLU, and the output layer uses $W^{(2)} \in \mathbb{R}^{4\times 1}$, bias $b^{(2)}$ $\mathbf{h}$ and sigmoid:

$$= \max\ 0, \mathbf{x}W^{(1)} + \mathbf{b}^{(1)} \quad , \tag{1}$$

$$\hat{y} = \sigma\ \mathbf{h}W^{(2)} + b^{(2)} \quad = \frac{1}{1+e^{\ -(\mathbf{h}W^{(2)}+b^{(2)})}}. \tag{2}$$

Each bias is a learnable offset, so a neuron can activate or stay silent regardless of the scale of its weighted input.

B. Loss Functions

We compared the squared error $L_{\text{MSE}} = (\ \ \hat{y}\ \ y)^2$ with the binary cross-entropy

$$L_{\text{BCE}} = \quad y \ln \hat{y} + (1\ \ y) \ln(1\ \ \hat{y})\ . \tag{3}$$

BCE gives stronger gradients when predictions are far from the label $y \in \{0, 1\}$, speeding convergence compared to MSE.

C. Gradients and Updates

With BCE and sigmoid, chain-rule backpropagation [3] gives $\delta^{(2)} = \hat{y}$ y and $\delta^{(1)} = \delta^{(2)}(W^{(2)})^{\top} \odot \mathbb{1}_{\{\mathbf{h}>0\}}$. With $\alpha = 0.01$:

$$W^{(2)} \leftarrow W^{(2)} \quad \alpha\, \mathbf{h}^{\top}\delta^{(2)}, \qquad b^{(2)} \leftarrow b^{(2)} \quad \alpha\, \delta^{(2)}, \tag{4}$$

$$W^{(1)} \leftarrow W^{(1)} \quad \alpha\, \mathbf{x}^{\top}\delta^{(1)}, \qquad \mathbf{b}^{(1)} \leftarrow \mathbf{b}^{(1)} \quad \alpha\, \delta^{(1)}. \tag{5}$$

III. Implemented Features

1) Three features (`perimeter3`, `area3`, `concavity3`) were standardized, with binary target mapping.
2) Biases ($\mathbf{b}^{(1)} = \mathbf{0}_4 b^{(2)} = 0$) were added to each layer's affine transformation.
3) A numerically stable BCE was implemented, with clipping ($\varepsilon = 10^{-12}$).
4) Backpropagation updates both weights and biases over 100 epochs.

IV. Experimental Results

On the first sample ($\hat{y} = 0.9846$, $y = 1$) before training, MSE is 0.000236 and BCE is 0.015482. Online gradient descent over 100 epochs gave the mean BCE loss in Table I. The final biases are $b^{(2)} = \quad 4.3554$ and $\mathbf{b}^{(1)} = [0.7641, 0.6550, {}^{0}.4216, 1.2785]$.

TABLE I
Mean BCE Training Loss

Epoch	0	10	20	30	40	50	60	70	80	90
Loss	.4053	.1564	.1491	.1475	.1466	.1455	.1449	.1446	.1444	.1442

V. Limitations

1) A fixed learning rate ($\alpha = 0.01$) can slow optimization on plateaus.
2) Instance-by-instance updates cost more than mini-batch vectorization.
3) Only training loss is reported; accuracy on unseen data is not yet measured.

VI. Upcoming Improvements

Bias vectors and BCE loss give smooth convergence. Next (Week 4) we implement optimizers such as Momentum and Adam, followed by a framework baseline comparison, model evaluation and transformers (Weeks 5–6).

References

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[2] W. H. Wolberg, W. N. Street, and O. L. Mangasarian, "Machine learning techniques to diagnose breast cancer from fine-needle aspirates," *Cancer Letters*, vol. 77, no. 2–3, pp. 163–171, 1994.

[3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.